

VeriScan AI - Project Modules, Files & Technical Architecture

Intelligent Deepfake Detection & Multimedia Verification Platform

PROJECT TECHNICAL SPECIFICATION & MODULE MAP

This document presents the complete file mapping, primary underlying algorithms, machine learning models, and forensic techniques for all 17 detection, analysis, and support modules in the VeriScan AI Deepfake Detection Platform.

Module 1: Image Deepfake Detection Module

Files Involved: [detection/views.py \(analyze_image\)](#)
[detection/ml.py \(predict_image, get_model\)](#)
[ml_model/best_image_deepfake_model.keras](#)

Technique: Deep Learning Convolutional Neural Network (CNN / MobileNetV2 architecture). The image is normalized and passed through the trained Keras binary classifier to predict REAL vs FAKE probability with high-precision confidence scoring.

Module 2: Error Level Analysis (ELA) Image Forensics

Files Involved: [detection/views.py \(analyze_image\)](#)
[detection/forensics.py \(generate_ela_image\)](#)

Technique: Digital Compression Forensics. Re-compresses the image at 90% JPEG quality, computes pixel-wise absolute difference against the original, and amplifies scale variances to highlight localized digital editing or face-swapping manipulation.

Module 3: Grad-CAM Heatmap Explainable AI (XAI)

Files Involved: [detection/views.py \(analyze_image\)](#)
[detection/forensics.py \(generate_gradcam\)](#)

Technique: Gradient-weighted Class Activation Mapping (Grad-CAM). Computes gradients of the target class score with respect to the final convolutional layer feature maps, producing a color-coded heatmap overlay showing exact spatial regions the model focused on.

Module 4: Video Deepfake & Anomaly Detection Module

Files Involved: [detection/views.py \(analyze_video\)](#)
[detection/video_ml.py \(get_video_metadata, extract_middle_frame, predict_frame\)](#)
[detection/video_forensics.py \(forensic_video_analysis, generate_video_gradcam\)](#)

Technique: Multi-Frame Neural Sampling + Spatial-Temporal Variance Analysis. Extracts representative frames via OpenCV, runs Haar-cascade face detection, evaluates frame-by-frame deep learning predictions, and analyzes temporal stability in brightness, sharpness, and noise.

Module 5: Synthetic Audio & Voice Clone Detection Module

Files Involved: [detection/views.py \(analyze_audio\)](#)
[detection/audio_ml.py \(audio_to_spectrogram, predict_audio, get_audio_metadata\)](#)
[detection/audio_forensics.py \(generate_waveform_image, generate_spectrogram_image\)](#)

Technique: Acoustic Spectrogram Analysis + CNN Classification. Converts audio waveforms into Mel-Frequency Spectrograms using Librosa / SciPy short-time Fourier transforms (STFT), analyzing pitch variation, high-frequency spectral roll-off, and neural voice synthesis signatures.

VeriScan AI - Project Modules, Files & Technical Architecture

Intelligent Deepfake Detection & Multimedia Verification Platform

Module 6: AI-Generated Text & NLP Detection Module

- Files Involved:** [detection/views.py \(analyze_text\)](#)
[detection/text_ml.py \(analyze_text_metadata, predict_text, get_top_contributing_words\)](#)
- Technique:** Natural Language Processing (NLP) + TF-IDF Vectorization & Scikit-Learn Classifier. Evaluates vocabulary richness, sentence length variance, burstiness, and word n-gram probability distributions to identify AI vs Human authorship.

Module 7: Document Forensics & Revision Tracking Module

- Files Involved:** [detection/views.py \(analyze_document\)](#)
[detection/document_forensics.py \(analyze_pdf_metadata, ocr_page, run_ela_on_page, detect_copy_move\)](#)
- Technique:** PyPDF2 Metadata Extraction + Tesseract OCR Text Inspection + Copy-Move Forgery Detection. Analyzes PDF incremental update trailers, digital signatures, producer software history, and page-level block duplication heuristics.

Module 8: Phishing & Malicious Link Safety Checker

- Files Involved:** [detection/views.py \(analyze_link\)](#)
[detection/link_checker.py \(analyze_url\)](#)
- Technique:** Domain Reputation & Heuristic Security Analysis. Performs URL structure parsing, WHOIS domain age checks, SSL certificate validation, IP reputation lookup, and suspicious TLD/redirect chain evaluation.

Module 9: Side-by-Side Dual File Comparison Engine

- Files Involved:** [detection/views.py \(compare_files_view\)](#)
[detection/file_compare.py \(compare_files, compare_images, compare_texts\)](#)
- Technique:** Structural Similarity Index (SSIM) + DiffLib Text Delta + Hash Verification. Performs pixel-level MSE/SSIM matrix comparison for images, and sequence matcher line-by-line diff analysis for text files.

Module 10: Batch Upload & Parallel Verification Engine

- Files Involved:** [detection/views.py \(analyze_batch\)](#)
[detection/ml.py, video_ml.py, audio_ml.py, text_ml.py](#)
- Technique:** Batch Orchestration & Multi-Format Dispatcher. Loops through multi-file zip/folder uploads, automatically identifying file MIME types and routing each file to its respective model pipeline.

Module 11: Live AI Web Proctoring & Integrity System

- Files Involved:** [detection/views.py \(proctor_session, proctor_analyze_frame\)](#)
[detection/proctor_engine.py \(analyze_frame_integrity\)](#)
[templates/detection/proctor_session.html](#)
- Technique:** Real-Time OpenCV Object & Multi-Person Detection. Analyzes live browser webcam frames via JavaScript Canvas API, flagging secondary persons, unauthorized smartphones/notes, off-screen gaze angle, and ambient noise spikes.

Module 12: Agentic AI Co-Pilot & Conversational Assistant

- Files Involved:** [detection/views.py \(copilot_chat\)](#)
[detection/agentic_core.py \(CopilotAgent, dispatch_analysis\)](#)
- Technique:** Rule-Based Intent Classifier + Dynamic Tool Dispatcher. Processes natural language user queries, automatically selecting diagnostic tools and generating plain-language explanation summaries.
-

VeriScan AI - Project Modules, Files & Technical Architecture

Intelligent Deepfake Detection & Multimedia Verification Platform

Module 13: QR-Verified Authenticity Certificate Generator

Files Involved: [detection/views.py](#) ([download_certificate](#))
[detection/certificate.py](#) ([compute_file_hash](#), [generate_qr_code](#), [build_certificate_pdf](#))

Technique: Cryptographic Hashing (SHA-256) + Dynamic QR Code Generation + FPDF Vector Layout. Embeds file cryptographic hashes and tamper-proof verification URLs into downloadable PDF certificates.

Module 14: Automated PDF Forensic Report Generation

Files Involved: [detection/views.py](#) ([download_report](#))
[detection/reports.py](#) ([build_image_report](#), [build_video_report](#), [build_audio_report](#), [build_text_report](#),
[build_document_report](#), [build_link_report](#), [build_compare_report](#), [build_proctor_report](#))

Technique: FPDF Vector Document Synthesis + Unicode Text Normalization. Dynamically compiles metadata, forensic charts, ELA/Grad-CAM images, and risk scores into standardized PDF reports.

Module 15: Job & Recruiter Profile Credibility Verifier

Files Involved: [detection/views.py](#) ([verify_job](#), [verify_recruiter](#))
[detection/job_verifier.py](#) ([search_and_verify_jobs](#), [verify_recruiter_profile](#))

Technique: Regex Pattern Matching + Heuristic Credibility Scoring. Scans job posting text and recruiter profiles for suspicious contact info, domain mismatches, upfront payment requests, and non-corporate emails.

Module 16: User Authentication & 2-Step OTP Verification

Files Involved: [accounts/views.py](#) ([register](#), [login](#), [verify_otp](#), [logout](#))
[accounts/models.py](#) (User)
[accounts/utils.py](#) ([send_otp_email](#))

Technique: Django AbstractUser Extension + Email Multi-Factor Authentication. Generates 6-digit cryptographically secure temporary OTP codes sent via Django SMTP backend for account activation.

Module 17: Administrative Monitoring & Telemetry Dashboard

Files Involved: [adminpanel/views.py](#) ([dashboard](#), [user_list](#), [system_logs](#))
[adminpanel/models.py](#)
[templates/adminpanel/dashboard.html](#)

Technique: Django ORM Aggregation & Analytics Telemetry. Computes real-time system usage statistics, media breakdown charts, user quota tracking, and audit log monitoring.
